35. HTTP Servers and Clients (net/http)

35. HTTP Servers and Clients (net/http)

Phase: Phase 6 — Systems and infrastructure
Estimated time: 2 days
Slug: `35-http-net`

Learning objectives

- Build handlers with net/http
- Use http.Client with timeouts
- Understand middleware chaining
- Serve JSON APIs safely

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

net/http is production-grade in the standard library. Understanding handlers, clients, and timeouts is baseline for Go services.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- http.HandleFunc / ServeMux route requests.
- Handlers write to ResponseWriter; read from Request.
- Clients should set Timeouts — never use bare default in prod.
- context on Request cancels when clients disconnect.
- ListenAndServe blocks; shutdown gracefully later lessons.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-http && cd /tmp/gw-http
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import (
    "fmt"
    "net/http"
    "net/http/httptest"
)

func health(w http.ResponseWriter, r *http.Request) {
    w.WriteHeader(http.StatusOK)
    fmt.Fprint(w, "ok")
}

func main() {
    req := httptest.NewRequest(http.MethodGet, "/healthz", nil)
    rr := httptest.NewRecorder()
    health(rr, req)
    fmt.Println(rr.Code, rr.Body.String())
}
```

Run it:

```
go run .
```

Expected output:

```
200 ok
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Write /healthz returning 200 ok.
2. httptest a handler without listening on a port.
3. Set http.Client{Timeout: 2*time.Second}.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Server

```
mux := http.NewServeMux()
mux.HandleFunc("GET /health", func(w http.ResponseWriter, r *http.Request) {
    w.WriteHeader(http.StatusOK)
    _, _ = w.Write([]byte("ok"))
})
srv := &http.Server{Addr: ":8080", Handler: mux, ReadHeaderTimeout: 5 *
time.Second}
log.Fatal(srv.ListenAndServe())
```

Always set timeouts. Infinite hangs are outages.

Client

```
client := &http.Client{Timeout: 10 * time.Second}
resp, err := client.Get(url)
```

Reuse clients; do not create one per request. Close response bodies.

Context

`r.Context()` cancels when the client disconnects — respect it in handlers.

Failure modes to watch while coding

- Default client with no timeout
- Ignoring resp.Body.Close()
- Blocking handler work without goroutine limits

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- Flask/FastAPI vs net/http — stdlib is enough for many services.
- requests defaults vs explicit Client timeouts.
- WSGI sync model vs Go's per-request goroutines.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would "work". The goal is a Go-shaped intuition.

Going deeper

Always set timeouts on http.Client in production. Prefer httptest for handler unit tests so CI needs no open ports.

Practice checklist

Before the exercise, confirm you can:

1. Explain "HTTP Servers and Clients (net/http)" aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson's rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Default client with no timeout
- Ignoring resp.Body.Close()
- Blocking handler work without goroutine limits

Exercises

Work in `exercises/35-http-net/`. Read that folder's `README.md` before you code.

What you will build

Implement a health handler (or client helper) so tests get 200/ok bodies.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/35-http-net/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. Why ReadHeaderTimeout?
2. How reuse http.Client?
3. What cancels r.Context()?

Further reading

- https://pkg.go.dev/net/http